README tasks and bug verification

57_match_three_gem · Claude Code session export

USER

Read the README file in the project repository. Note down all the tasks present in it.
The first bug has been fixed, but before moving ahead, i want you to verify once if the fix was correct in the latest commit.
For the remaining tasks i want you to list them and add the features one by one, not all at once.
Before moving on to the next feature, i want you to test the code for errors and then let me check it once for functional integrity.

CLAUDE

```
Ran: List files, read README, show latest commit
Ran: Read all source files and prior board.py commit
Ran: Inspect earlier board.py history for drop_and_refill
Ran: Run headless test of Task 1 fix over 300 boards
Ran: Implement cascade combo multiplier in board and engine
```

Task 2 code is in. The diff touched every line, so I'm checking for a line-ending change.

```
Ran: Compare line endings of original and edited files
Ran: Restore CRLF line endings and show diff
```

The last line of `game_engine.py` picked up a stray `\r`. Removing it, then running the tests.

```
Ran: Fix EOF byte and run combo + regression tests
```

Task 1's fix is correct, and Task 2 (the cascade combo multiplier) is implemented and tested. I haven't committed anything. It's ready for you to play-test.

The tasks in the README

#	Task	Status
1	Invalid swaps should not cost a move	Fixed in `98f48a0`, checked
2	Cascade combo multiplier	Implemented, waiting for your check
3	4-in-a-row special gems that clear a whole row or column	Not started
4	Hint that highlights a valid swap after 5 seconds without input	Not started

Task 1 check

The fix in board.py:154 is right: a move is only taken when the swap makes a match, and a swap with no match is reversed. I tested it on 300 random boards:

- Swaps with no match leave the move count and the board unchanged.
- Swaps between gems that aren't next to each other are rejected.
- Swaps that make a match use one move and add score.

Something else that commit fixed: it also added the missing `for c in range(GRID_SIZE):` loop in `drop_and_refill`. Without that line, the game would crash with a `NameError` the first time

gems needed to fall and refill, which can even happen while the opening board is being built. The commit message doesn't mention this.

Task 2: what changed

- **board.py** `resolve_matches`**:** each round of falling matches in a turn gets a higher multiplier. The first round is worth ×1, the next ×2, then ×3, and so on (10 points per gem × the multiplier). The function now returns points instead of a count of cleared gems, and saves the turn's chain length in `last_combo`.
- **game_engine.py:** when a chain reaches ×2 or more, "COMBO xN!" shows in gold under the score bar for 1.5 seconds.

Testing

- **Scoring:** a fixed sequence of three matches gave the expected 30 + 80 + 90 = 200 points.
- **Random games:** 200 games ran to the end with no errors. Combos happened about 30% of the time, and the longest chain was ×7.
- **Drawing and restart:** the screen draws correctly with no display attached, and restarting works.
- **Task 1 regression:** the Task 1 test still passes.

My first edit changed the files' line endings, so I converted them back to Windows-style (CRLF). The diff now shows only the real changes.

To check it yourself:

```
python main.py
```

Score should now go up faster when chains happen. The gold COMBO text should appear only for chains of ×2 or more, not for a single match.

Let me know whether it works for you. Once you approve, I'll commit Task 2 and start Task 3.

USER

works fine, i committed and pushed, now move on to task 3.

CLAUDE

Starting Task 3. My plan:

- **Spawning:** a run of 4 or more turns one of its gems into a glowing special gem instead of clearing it. If the gem you swapped is in the run, that's the one that changes; otherwise it's a gem near the middle.
- **Direction:** a horizontal run makes a row-clearer, and a vertical run makes a column-clearer.
- **Detonation:** when a special gem is matched later, it clears its whole row or column. If that line contains another special gem, it goes off too.

The files use Windows (CRLF) line endings, so I'll convert them to plain LF while editing and switch them back afterward.

```
Ran: Check clean state and convert files to LF for editing
Read game/board.py
```

Now making the edits: gem state, run detection, spawning and detonation, then rendering.

```
Ran: Implement special line-clear gems in board.py
```

Compiles. Now the tests: set up boards by hand for spawning, detonation and chain reactions, plus random games to catch crashes.

```
Ran: Run special gem tests plus regression tests
Ran: Debug test 4 board layout
Ran: Fix test fixtures and rerun all tests
Ran: Rerun tests with corrected expectations
Ran: Crop and convert render snapshot to PNG
Read /tmp/claude-1000/-home-nishit-Downloads-Temp-57-match-three-gem/ecfcd3a3-38fc-46e7-
9c17-a441d7a9b638/scratchpad/crop.png
Ran: Restore CRLF endings, verify diff and recompile
```

Task 3 (the 4-in-a-row special gems) is in and all my tests pass, including the Task 1 and Task 2 checks. Only board.py changed. I haven't committed it; it's ready for you to try.

How it works

- **Making a special gem:** matching 4 or more in a line turns one of those gems into a special gem instead of clearing it. If the gem you swapped is in the line, that's the one that changes; otherwise it's one near the middle. A 5-in-a-row still makes just one special gem.
- **Row or column:** a horizontal line of 4 makes a row-clearer, and a vertical one makes a column-clearer. The README didn't say which should be which, so I chose this pairing; it's easy to flip.
- **Detonation:** when a special gem is part of a later match, it clears its entire row or column. If that line contains another special gem, that one goes off too. A brand-new special gem is never destroyed in the same step it's created.
- **Score:** every cleared gem counts, including gems cleared by a detonation, and the Task 2 combo multiplier still applies.
- **Appearance:** special gems have a pulsing glowing border with stripes: horizontal for row-clearers, vertical for column-clearers. The cleared line flashes white for 0.4 s.
- **Starting board:** a new or restarted board never begins with special gems.

How the code changed

- **Run detection:** I replaced the old 3-gem window check with `find_runs()`, which finds whole runs and their length. I needed this to tell a 4-run from two overlapping 3-runs. `find_matches()` still works the same way: on 2000 random boards it gave exactly the same results as the old version.
- **New helpers:** `pick_special_spot()`, `expand_detonations()` and `render_special()`.

Testing

- **Hand-built boards:** 7 of these checked the swapped-gem placement, both directions, the 5-run case, row and column detonation, one special setting off another, and the new special gem surviving.
- **Random games:** 300 games ran to the end with no crash, a full board after every move, and no leftover matches. About 224 special gems were created and 65 went off.
- **Drawing:** I drew the board with special gems and flashes on screen and looked at the image.

Two test failures along the way were mistakes in my test setups, not in the game. In one, my hand-built board accidentally created an extra match, so the extra 30 points were correct.

To try it:

```
python main.py
```

Try lining up 4 of a colour, then match the glowing gem to set it off.

Let me know how it plays. Once you've committed it, I'll move on to Task 4, the idle hint.

USER

works fine, committed and pushed, move on to task 4

CLAUDE

Starting Task 4. My plan:

- **Finding a hint:** the board checks each pair of neighbouring gems, swaps them in memory, and keeps the first pair that would make a match.
- **Idle timer:** the game tracks the last time you clicked or pressed R. Falling-gem animations also restart the timer, so the 5 seconds count from when the board stops moving.
- **Showing it:** once the timer passes 5 seconds, both gems in the pair get a pulsing gold outline. Any input clears it.

```
Ran: Check state, convert to LF, view engine and anchors
Ran: View board render section
Ran: Implement idle hint in board and engine
```

One problem: if the board has no valid move, `find_hint()` returns `None` and would run again every frame. I'll add a flag so it searches only once per idle period.

```
Ran: Add once-per-idle guard and run hint + regression tests
```

All passing. Now I'll restore the CRLF line endings, check the diff, and save a screenshot with the hint showing.

```
Ran: Restore CRLF, verify diff, capture hint screenshot
Read /tmp/claude-1000/-home-nishit-Downloads-Temp-57-match-three-gem/ecfcd3a3-38fc-46e7-
9c17-a441d7a9b638/scratchpad/hint.png
```

Task 4 (the idle hint) is in, and all my tests pass along with the earlier Task 1–3 checks. I haven't committed it; it's ready for you to try.

How it works

- **When it shows:** after 5 seconds with no input, two neighbouring gems that can be swapped to make a match get a pulsing gold outline and a soft white shimmer.
- **What counts as input:** any click (even outside the board) or pressing R hides the hint and restarts the 5-second countdown.
- **Falling gems:** the countdown doesn't start until gems stop falling, so the hint never appears mid-cascade. It's also hidden once the game is over.
- **No moves left:** the board checks for a move only once per idle period. If there isn't one, no hint appears and it doesn't keep re-checking every frame.

What changed

- **board.py**:
  - `find_hint()` tries every pair of neighbouring gems, swapping each pair in and back out to see if it would make a match.
  - `render_hint()` draws the pulsing outline and shimmer.
  - A new `hint` field is cleared on restart.
- **game_engine.py**:
  - `HINT_DELAY_MS = 5000` sets the delay; change it if you want the hint faster or slower.
  - `register_input()` restarts the countdown and hides the hint.
  - `update()` handles the timing.

Testing

- **Finding hints:** on 500 random boards, every hint was a real neighbouring pair that makes a match, and the search never changed the board. On a board with no possible moves it correctly returned nothing.
- **Timing:** using a simulated clock, I checked that:
  - nothing appears at 4 s and the hint appears after 5 s;
  - a click hides it and restarts the 5 seconds;
  - falling gems hold the countdown;
  - there's no hint after the game ends;
  - restarting clears it;
  - the search runs once per idle period.
- **Visual:** I captured a screenshot with the hint showing. It highlighted a blue gem and the orange gem below it; swapping them lines the orange up with two others.

To try it:

```
python main.py
```

Wait about 5 seconds without clicking.

This was the last task in the README. Once you've confirmed it plays well and committed, all four are done. Your submission still needs the before and after videos.

USER

the hint may seem unnoticeable because its very light. please make the hints have more contrast

CLAUDE

The current hint is a thin gold border plus a faint white overlay, which barely shows up on yellow and orange gems. Making it stronger:

- a dark outline around the outside so it stands out from any gem colour;
- a thicker border that pulses between bright gold and white and grows slightly with each pulse;
- a stronger white shimmer over the gem.

```
Ran: Strengthen hint visuals, rerun tests, capture pulse extremes
Read /tmp/claude-1000/-home-nishit-Downloads-Temp-57-match-three-gem/ecfcd3a3-38fc-46e7-9c17-a441d7a9b638/scratchpad/hint_0.png
Read /tmp/claude-1000/-home-nishit-Downloads-Temp-57-match-three-gem/ecfcd3a3-38fc-46e7-9c17-a441d7a9b638/scratchpad/hint_1.png
```

The hint is much more visible now. The only change is `render_hint()` in board.py; all the earlier tests still pass.

	Before	Now
Border	2–4 px gold	5 px, pulsing between bright gold and white
Outline	none	3 px black ring around the border
Size	fixed	grows a few pixels on each pulse
White shimmer over the gem	30–100 opacity	60–160 opacity
Pulse speed	200 ms	180 ms, a little faster

I saved screenshots at both ends of the pulse:

- **At its dimmest:** a thick gold ring. It even shows up on a yellow gem, because the black outline separates it from the gem colour.
- **At its brightest:** a white ring with a strong white wash over the gem. The two highlighted gems stand out clearly from everything around them.

Your Task 4 changes are still staged but not committed. This contrast change is unstaged on top of them, so you can commit them together or separately.

To check it:

```
python main.py
```

Wait about 5 seconds without clicking.

If it's still too subtle or now too strong, I can tune the line thickness, how much it grows, or the colours.